

ASSIGNMENT - 1 CO 1

NAME: LAKSHA.M P

REG NO:1924260302

COURSE CODE:CSA0824

SUBJECT NAME: PYTHON PROGRAMMING

SLOT:D

GITHUB REPOSITORY:

Task 1 – Data Analysis (supporting code, no TODO given)

Aim: To identify the inputs, outputs and rules required to calculate milk payments.

Algorithm:

1. Start.
2. Define the base rate, standard fat, standard SNF and step values.
3. Store the farmer details, milk quantity, fat and SNF values.
4. Calculate the fat and SNF step differences.
5. Calculate the rate per litre.
6. Calculate the payment for each farmer.
7. Add each payment to the total.
8. Display the rate, payment and total payment.
9. Stop.

Input:

TASK 1.py - C:/Users/abina/AppData/Local/Programs/Python/Python314/TASK 1.py (3.14.7)

File Edit Format Run Options Window Help

```
BASE_RATE = 30.00
FAT_STEP_VALUE = 0.40
SNF_STEP_VALUE = 0.30
DAYS = 10

farmer_id = 101
litres = 10.5
fat = 4.2
snf = 8.7
shift = "Morning"

fat_steps = round((fat - 3.5) / 0.1)
snf_steps = round((snf - 8.5) / 0.1)

rate = BASE_RATE + fat_steps * FAT_STEP_VALUE + snf_steps * SNF_STEP_VALUE
amount = round(litres * rate, 2)
ten_day_total = round(amount * DAYS, 2)

print("Farmer ID:", farmer_id)
print("Litres:", litres)
print("Fat %:", fat)
print("SNF %:", snf)
print("Shift:", shift)
print("Rate per litre: Rs.", round(rate, 2))
print("Amount: Rs.", amount)
print("Ten-day total: Rs.", ten_day_total)
print("Fat valid:", 2.0 <= fat <= 7.0)
print("Litres valid:", litres >= 0)
```

Output:

IDLE Shell 3.14.7

File Edit Shell Debug Options Window Help

```
Python 3.14.7 (tags/v3.14.7:823f032, Aug  5 2026, 10:51:32) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
=== RESTART: C:/Users/abina/AppData/Local/Programs/Python/Python314/TASK 1.py ==
Farmer ID: 101
Litres: 10.5
Fat %: 4.2
SNF %: 8.7
Shift: Morning
Rate per litre: Rs. 33.4
Amount: Rs. 350.7
Ten-day total: Rs. 3507.0
Fat valid: True
Litres valid: True
>>>
```

Task 2 — Data Types, Operators and Decimal

Aim: To demonstrate Python data types, arithmetic operators and Decimal calculations.

Algorithm:

1. Start.
2. Import the Decimal class.
3. Create variables of different data types.
4. Display the types of the variables.
5. Perform arithmetic operations using /, // and %.
6. Compare floating-point arithmetic with Decimal arithmetic.
7. Display the results.
8. Stop.

PROGRAM

Input:

task 2.py - C:/Users/abina/AppData/Local/Programs/Python/Python314/task 2.py (3.14.7)

File Edit Format Run Options Window Help

```
from decimal import Decimal

print(type(10))
print(type(10.0))
print(type("10"))
print(type(True))
print(type([10, 8.5]))

print("7 / 2 =", 7 / 2)
print("7 // 2 =", 7 // 2)
print("7 % 2 =", 7 % 2)
print("-7 // 2 =", -7 // 2)

print("Float comparison:", 0.1 + 0.2 == 0.3)
print("Decimal comparison:", Decimal("0.1") + Decimal("0.2") == Decimal("0.3"))

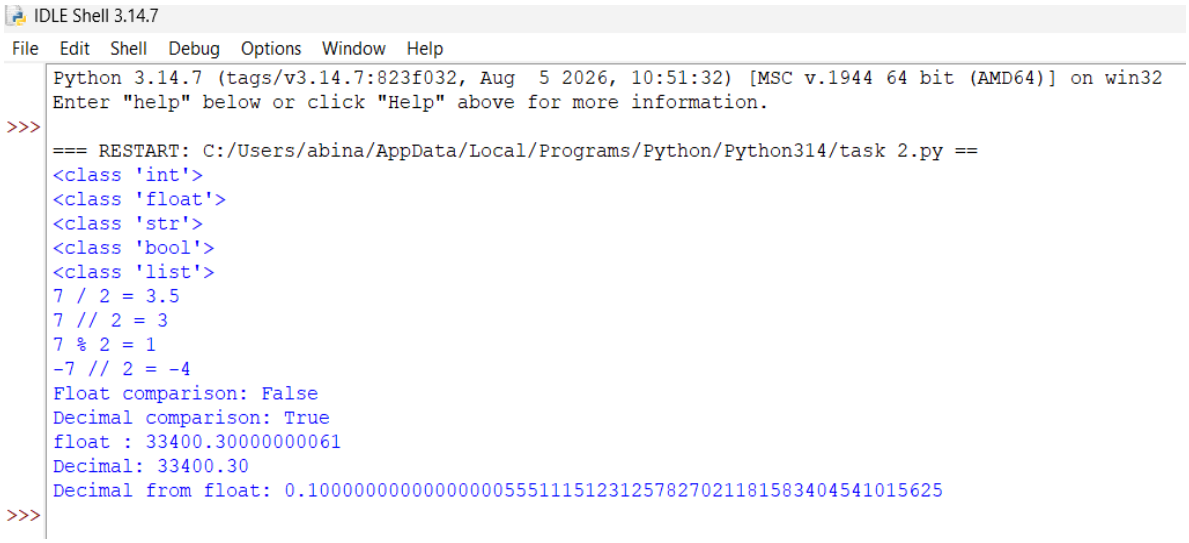
payments = ["33.40"] * 1000 + ["0.10"] * 3

def total_float(items):
    total = 0.0
    for amount in items:
        total += float(amount)
    return total

def total_decimal(items):
    total = Decimal("0.00")
    for amount in items:
        total += Decimal(amount)
    return total

print("float :", total_float(payments))
print("Decimal:", total_decimal(payments))
print("Decimal from float:", Decimal(float("0.1")))
```

Output:



Task 3 — Variables, Constants and Assignment

Aim: To demonstrate variables, constants, dynamic typing and assignment operations in Python.

Algorithm:

1. Start.
2. Define the base rate and step values as constants.
3. Create a variable and assign an integer value.
4. Display the variable and its type.
5. Assign a string value to the same variable.
6. Swap the values of two variables.
7. Create a dictionary containing the rate details.
8. Display the results.
9. Stop.

Input:

TASK 3.py - C:/Users/abina/AppData/Local/Programs/Python/Python314/TASK 3.py (3.14.7)

File Edit Format Run Options Window Help

```
BASE_RATE = 30.00
FAT_STEP_VALUE = 0.40
SNF_STEP_VALUE = 0.30
MIN_FAT = 2.0
MAX_FAT = 7.0

farmer_id = 101
litres = 10
fat = 4.2
snf = 8.7

print("Base rate:", BASE_RATE)
print("Fat step value:", FAT_STEP_VALUE)
print("SNF step value:", SNF_STEP_VALUE)
print("Fat limits:", MIN_FAT, "to", MAX_FAT)

value = 10
print("Before:", value, type(value))
value = "ten"
print("After:", value, type(value))

a = 5
b = 10
a, b = b, a
print("After swapping:", a, b)
```

Output:

IDLE Shell 3.14.7

File Edit Shell Debug Options Window Help

```
Python 3.14.7 (tags/v3.14.7:823f032, Aug  5 2026, 10:51:32) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
=== RESTART: C:/Users/abina/AppData/Local/Programs/Python/Python314/TASK 3.py ==
Base rate: 30.0
Fat step value: 0.4
SNF step value: 0.3
Fat limits: 2.0 to 7.0
Before: 10 <class 'int'>
After: ten <class 'str'>
After swapping: 10 5
>>>
```

Task 4 — Expressions and Rounding

Aim: To calculate the milk rate and payment using arithmetic expressions and rounding operations.

Algorithm:

1. Start.
2. Read the milk quantity, fat percentage and SNF percentage.
3. Calculate the fat step difference.
4. Calculate the SNF step difference.
5. Calculate the rate per litre using the base rate and step values.
6. Calculate the total payment by multiplying quantity by rate.
7. Apply rounding where required.
8. Display the step differences, rate and payment.
9. Stop.

Input:

task 4.py - C:/Users/abina/AppData/Local/Programs/Python/Python314/task 4.py (3.14.7)

File Edit Format Run Options Window Help

```
import math

BASE_RATE = 30.00
FAT_STEP_VALUE = 0.40
SNF_STEP_VALUE = 0.30

def calculate_rate(fat, snf):
    fat_steps = round((fat - 3.5) / 0.1)
    snf_steps = round((snf - 8.5) / 0.1)
    rate = BASE_RATE + fat_steps * FAT_STEP_VALUE + snf_steps * SNF_STEP_VALUE
    return round(rate, 2)

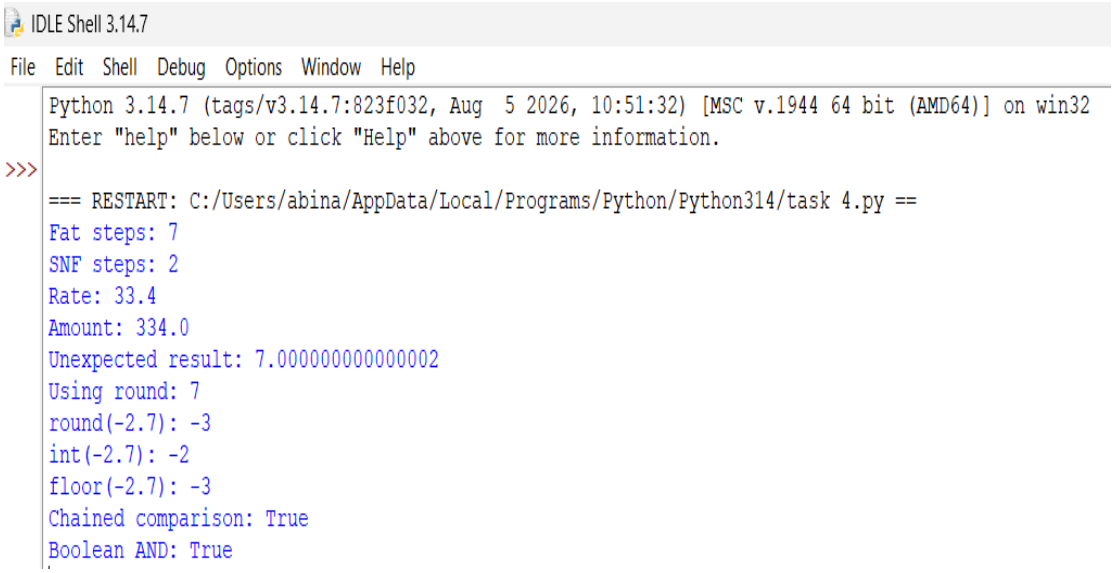
fat = 4.2
snf = 8.7
litres = 10

print("Fat steps:", round((fat - 3.5) / 0.1))
print("SNF steps:", round((snf - 8.5) / 0.1))
print("Rate:", calculate_rate(fat, snf))
print("Amount:", round(litres * calculate_rate(fat, snf), 2))

print("Unexpected result:", (4.2 - 3.5) / 0.1)
print("Using round:", round((4.2 - 3.5) / 0.1))
print("round(-2.7):", round(-2.7))
print("int(-2.7):", int(-2.7))
print("floor(-2.7):", math.floor(-2.7))

print("Chained comparison:", 2.0 <= fat <= 7.0)
print("Boolean AND:", fat >= 2.0 and fat <= 7.0)
```

Output:



Task5 — User Input and Validation

Aim: To accept milk details from the user, validate the inputs and calculate the payment.

Algorithm:

1. Start.
2. Read litres, fat percentage and SNF percentage.
3. Check whether the milk quantity is negative.
4. Check whether fat is between 2.0 and 7.0.
5. Check whether SNF is non-negative.
6. If any input is invalid, display an error message.
7. Otherwise, calculate the fat and SNF step differences.
8. Calculate the rate and total payment.
9. Display the rate and payment.
10. Stop.

Input:

task 5.py - C:/Users/abina/AppData/Local/Programs/Python/Python314/task 5.py (3.14.7)

File Edit Format Run Options Window Help

```
def read_number(message):
    while True:
        text = input(message).strip()
        try:
            value = float(text)
            if value < 0:
                print("Enter a non-negative number.")
                continue
            return value
        except ValueError:
            print("Invalid input. Enter a valid number.")

farmer_id = input("Enter farmer ID: ").strip()
litres = read_number("Enter litres: ")
fat = read_number("Enter fat percentage: ")
snf = read_number("Enter SNF percentage: ")

if not farmer_id:
    print("Invalid farmer ID")
elif not 2.0 <= fat <= 7.0:
    print("Invalid fat percentage")
elif snf <= 0:
    print("Invalid SNF percentage")
else:
    rate = 30 + round((fat - 3.5) / 0.1) * 0.40
    rate += round((snf - 8.5) / 0.1) * 0.30
    amount = round(litres * rate, 2)

    print(f"Farmer ID: {farmer_id}")
    print(f"Rate per litre: Rs. {rate:.2f}")
    print(f"Amount: Rs. {amount:.2f}")
```

Output:

IDLE Shell 3.14.7

File Edit Shell Debug Options Window Help

```
Python 3.14.7 (tags/v3.14.7:823f032, Aug  5 2026, 10:51:32) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
=== RESTART: C:/Users/abina/AppData/Local/Programs/Python/Python314/task 5.py ==
Enter farmer ID: 101
Enter litres: 10
Enter fat percentage: 4.2
Enter SNF percentage: 8.7
Farmer ID: 101
Rate per litre: Rs. 33.40
Amount: Rs. 334.00
>>>
```

Task 6 — Functions and Return Values

Aim: To develop reusable functions for calculating the milk rate, individual payment and ten-day total.

Algorithm:

1. Start.
2. Define a function to calculate the rate per litre.
3. Define a function to calculate the payment.
4. Define a function to calculate the ten-day total.
5. Store the milk entries in a list.
6. Call the payment function for each entry.
7. Add the individual payments to the total.
8. Display the individual payments and ten-day total.
9. Stop.

PROGRAM CODE:

Input:

task 6.py - C:/Users/abina/AppData/Local/Programs/Python/Python314/task 6.py (3.14.7)

File Edit Format Run Options Window Help

```
BASE_RATE = 30.00
FAT_STEP_VALUE = 0.40
SNF_STEP_VALUE = 0.30

def rate_per_litre(fat, snf):
    """Calculate the rate per litre."""
    fat_steps = round((fat - 3.5) / 0.1)
    snf_steps = round((snf - 8.5) / 0.1)
    rate = BASE_RATE + fat_steps * FAT_STEP_VALUE
    rate += snf_steps * SNF_STEP_VALUE
    return round(rate, 2)

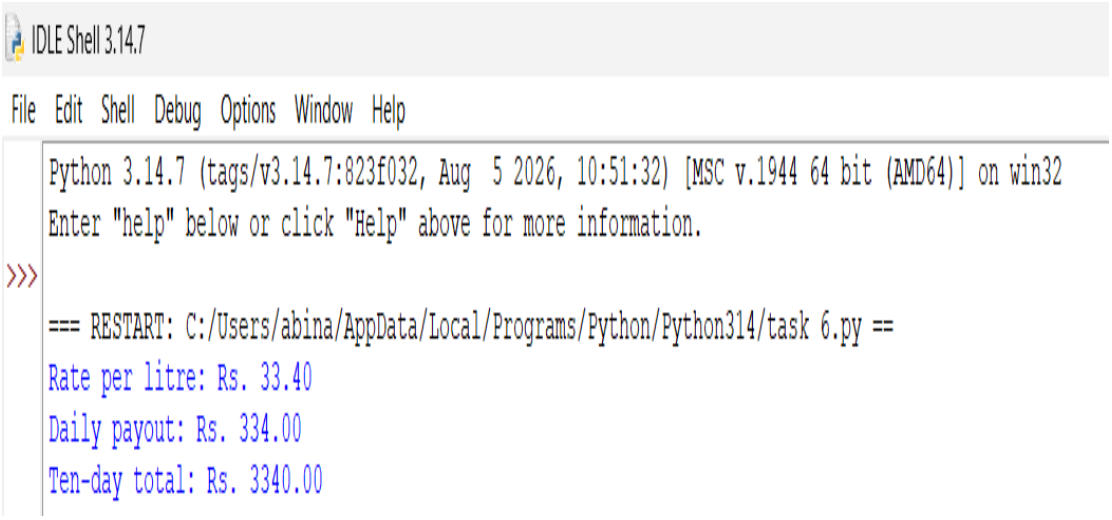
def payout(litres, fat, snf):
    """Calculate the payment for the milk supplied."""
    return round(litres * rate_per_litre(fat, snf), 2)

def ten_day_total(daily_amounts):
    """Calculate the total payment for ten days."""
    return round(sum(daily_amounts), 2)

rate = rate_per_litre(4.2, 8.7)
amount = payout(10, 4.2, 8.7)
total = ten_day_total([amount] * 10)

print(f"Rate per litre: Rs. {rate:.2f}")
print(f"Daily payout: Rs. {amount:.2f}")
print(f"Ten-day total: Rs. {total:.2f}")
```

Output:



Task 7 — Creating and Importing a Module

Aim: To create a Python module containing milk-rate functions and import it into another program.

Algorithm:

1. Start.
2. Create a file named milkrate.py.
3. Define the rate calculation function.
4. Define the payment calculation function.
5. Save the module.
6. Create another file named test_milkrate.py.
7. Import the required functions from milkrate.
8. Call the imported functions with sample values.
9. Display the rate and payment.
10. Stop.

Input:

task 7 (2).py - C:/Users/abina/AppData/Local/Programs/Python/Python314/task 7 (2).py (3.14.7)

File Edit Format Run Options Window Help

```
from decimal import Decimal
import math

BASE_RATE = 30.00
FAT_STEP_VALUE = 0.40
SNF_STEP_VALUE = 0.30


def rate_per_litre(fat, snf):
    """Calculate the rate per litre."""
    fat_steps = round((fat - 3.5) / 0.1)
    snf_steps = round((snf - 8.5) / 0.1)

    rate = BASE_RATE
    rate += fat_steps * FAT_STEP_VALUE
    rate += snf_steps * SNF_STEP_VALUE

    return round(rate, 2)


def payout(litres, fat, snf):
    """Calculate the amount payable."""
    return round(litres * rate_per_litre(fat, snf), 2)


# Display the rate table
print("Milk Co-operative Rate Table")
print("-" * 55)

entries = [
    (10, 3.5, 8.5),
    (8.5, 4.2, 8.7),
    (12, 3.1, 8.3)
]

for litres, fat, snf in entries:
    rate = rate_per_litre(fat, snf)
    pay = payout(litres, fat, snf)
    print(f"{litres:5} L fat {fat} snf {snf} rate {rate:.2f} pay {pay:.2f}")

# Module and standard library demonstration
print("-" * 55)
print("Square root of 25:", math.sqrt(25))
print("Exact decimal sum:", Decimal("0.1") + Decimal("0.2"))
print("Payout example:", payout(10, 4.2, 8.7))
```

Output :

IDLE Shell 3.14.7

File Edit Shell Debug Options Window Help

```
Python 3.14.7 (tags/v3.14.7:823f032, Aug  5 2026, 10:51:32) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/abina/AppData/Local/Programs/Python/Python314/task 7 (2).py
Milk Co-operative Rate Table
-------------------------------------------------------
   10 L fat 3.5 snf 8.5 rate 30.00 pay 300.00
  8.5 L fat 4.2 snf 8.7 rate 33.40 pay 283.90
   12 L fat 3.1 snf 8.3 rate 27.80 pay 333.60
-------------------------------------------------------
Square root of 25: 5.0
Exact decimal sum: 0.3
Payout example: 334.0
```

Task 8 — Algorithm, Pseudocode and Hand Tracing \

Aim: To implement and trace the milk-payment algorithm and calculate the running total.

Algorithm:

1. Start.
2. Initialise the total payment to zero.
3. Read or retrieve the milk entries.
4. Validate each entry.
5. Calculate the fat and SNF step differences.
6. Calculate the rate per litre.
7. Calculate the individual payment.
8. Add the payment to the running total.
9. Repeat until all entries are processed.
10. Stop.

Input:

task 8.py - C:/Users/abina/AppData/Local/Programs/Python/Python314/task 8.py (3.14.7)

File Edit Format Run Options Window Help

```
def calculate_payment(litres, fat, snf):
    """Calculate milk payment."""
    if litres < 0 or not 2.0 <= fat <= 7.0 or snf <= 0:
        return None

    fat_steps = round((fat - 3.5) / 0.1)
    snf_steps = round((snf - 8.5) / 0.1)

    rate = 30 + fat_steps * 0.40 + snf_steps * 0.30
    return round(litres * rate, 2)

entries = [
    (10, 3.5, 8.5),
    (8.5, 4.2, 8.7),
    (12, 3.1, 8.3)
]

total = 0

for litres, fat, snf in entries:
    amount = calculate_payment(litres, fat, snf)
    if amount is not None:
        print("Payment: Rs.", amount)
        total += amount
    else:
        print("Invalid entry")

print("Total payment: Rs.", round(total, 2))
print("Ten-day total: Rs.", round(total * 10, 2))
```

Output:

IDLE Shell 3.14.7

File Edit Shell Debug Options Window Help

```
Python 3.14.7 (tags/v3.14.7:823f032, Aug  5 2026, 10:51:32) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
=== RESTART: C:/Users/abina/AppData/Local/Programs/Python/Python314/task 8.py ==
Payment: Rs. 300.0
Payment: Rs. 283.9
Payment: Rs. 333.6
Total payment: Rs. 917.5
Ten-day total: Rs. 9175.0
```

Task 9 — Complete Calculator and Automated Testing

Aim: To develop a milk-payment calculator and test normal, boundary and invalid inputs.

Algorithm:

1. Start.
2. Define the rate calculation function.
3. Validate fat, SNF and milk quantity.
4. Calculate the rate and payment for valid inputs.
5. Write test cases for normal values.
6. Write test cases for minimum and maximum fat values.
7. Test invalid values using exception handling.
8. Use assertions to compare actual and expected results.
9. Display the test results.

10.Stop.

Input:

task 9.py - C:/Users/abina/AppData/Local/Programs/Python/Python314/task 9.py (3.14.7)

File Edit Format Run Options Window Help

```
    snf_steps = round((snf - 8.5) / 0.1)
    return round(BASE_RATE + fat_steps * FAT_STEP_VALUE
                 + snf_steps * SNF_STEP_VALUE, 2)

def payout(litres, fat, snf):
    if litres < 0:
        raise ValueError("Litres cannot be negative")
    if not 2.0 <= fat <= 7.0:
        raise ValueError("Fat must be between 2.0 and 7.0")
    if snf <= 0:
        raise ValueError("SNF must be positive")
    return round(litres * rate_per_litre(fat, snf), 2)

# Test cases
tests = [
    (10, 3.5, 8.5, 300.00),
    (8.5, 4.2, 8.7, 283.90),
    (12, 3.1, 8.3, 333.60),
    (10, 2.0, 8.5, 240.00),
    (10, 7.0, 8.5, 440.00),
]

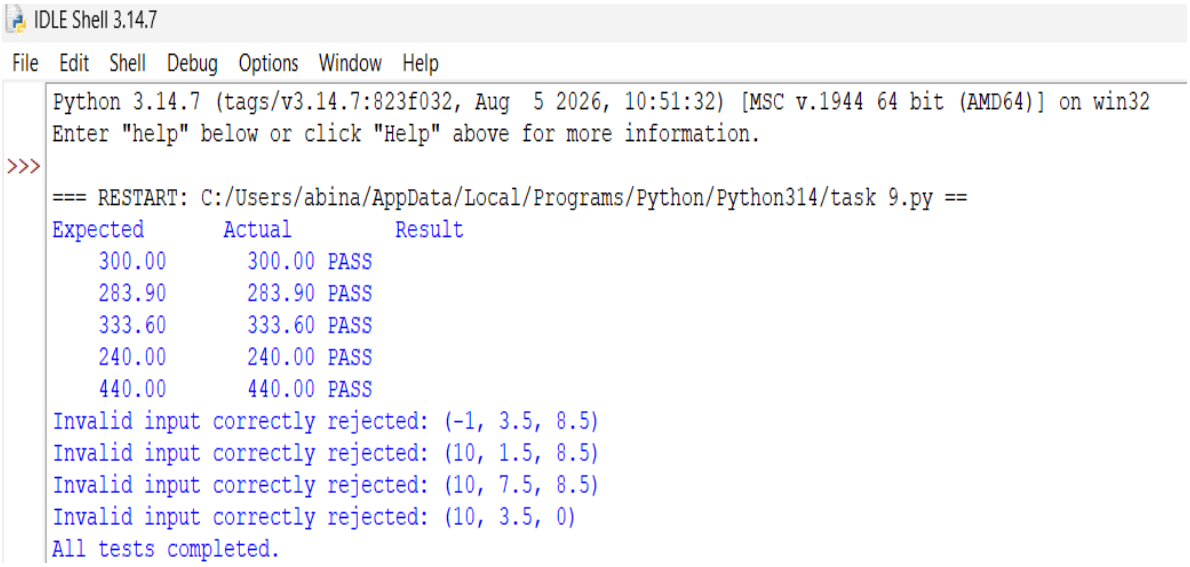
print("Expected       Actual         Result")
for litres, fat, snf, expected in tests:
    actual = payout(litres, fat, snf)
    result = "PASS" if actual == expected else "FAIL"
    print(f"{expected:10.2f} {actual:12.2f} {result}")
    assert actual == expected

# Invalid input tests
invalid_tests = [
    (-1, 3.5, 8.5),
    (10, 1.5, 8.5),
    (10, 7.5, 8.5),
    (10, 3.5, 0)
]

for case in invalid_tests:
    try:
        payout(*case)
    except ValueError:
        print("Invalid input correctly rejected:", case)
    else:
        print("FAIL:", case)

print("All tests completed.")
```

Output:



Task 10 — Performance Comparison: Float vs Decimal from decimal import Decimal

Aim: To compare the execution time and numerical results of float and Decimal calculations for multiple payments.

Algorithm:

1. Start.
2. Import Decimal and perf_counter.
3. Define a function to calculate totals using float.
4. Define another function to calculate totals using Decimal.
5. Create payment lists of different sizes.
6. Record the start time for float calculations.
7. Calculate the float total and elapsed time.
8. Repeat using Decimal.
9. Display both totals and execution times.
10. Stop.

Input:

task 10.py - C:/Users/abina/AppData/Local/Programs/Python/Python314/task 10.py (3.14.7)

File Edit Format Run Options Window Help

```
from decimal import Decimal
import time

sizes = [300, 6000, 150000]

for n in sizes:
    payments = ["33.40"] * n + ["0.10"] * 3

    start = time.perf_counter()
    float_total = sum(float(x) for x in payments)
    float_time = time.perf_counter() - start

    start = time.perf_counter()
    decimal_total = sum((Decimal(x) for x in payments),
                        Decimal("0.00"))
    decimal_time = time.perf_counter() - start

    print("Payments:", n)
    print("Float total:", round(float_total, 2))
    print("Decimal total:", decimal_total)
    print("Difference:", abs(Decimal(str(float_total)) - decimal_total))
    print("Float time:", round(float_time, 6), "seconds")
    print("Decimal time:", round(decimal_time, 6), "seconds")
    print()
```

Output:

IDLE Shell 3.14.7

File Edit Shell Debug Options Window Help

```
Python 3.14.7 (tags/v3.14.7:823f032, Aug  5 2026, 10:51:32) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
== RESTART: C:/Users/abina/AppData/Local/Programs/Python/Python314/task 10.py ==
Payments: 300
Float total: 10020.3
Decimal total: 10020.30
Difference: 0.00
Float time: 7.3e-05 seconds
Decimal time: 0.000173 seconds

Payments: 6000
Float total: 200400.3
Decimal total: 200400.30
Difference: 0.00
Float time: 0.001177 seconds
Decimal time: 0.002434 seconds

Payments: 150000
Float total: 5010000.3
Decimal total: 5010000.30
Difference: 0.00
Float time: 0.026577 seconds
Decimal time: 0.028097 seconds
```

Task 11 — Recommended Design from decimal import Demical

Aim: To develop a reliable milk-payment calculator using Decimal arithmetic, validation and reusable functions.

Algorithm:

1. Start.
2. Define named constants for the milk rate.
3. Create a function to calculate the rate using Decimal.
4. Validate the fat, SNF and quantity values.
5. Calculate the rate per litre.
6. Calculate the payment using Decimal arithmetic.
7. Round the payment to two decimal places.
8. Display the payment.
9. Stop.

Input:

task 11.py - C:/Users/abina/AppData/Local/Programs/Python/Python314/task 11.py (3.14.7)

File Edit Format Run Options Window Help

```
from decimal import Decimal

BASE_RATE = Decimal("30.00")
FAT_STEP_VALUE = Decimal("0.40")
SNF_STEP_VALUE = Decimal("0.30")

def rate_per_litre(fat, snf):
    """Calculate the rate per litre."""
    fat_steps = round((fat - 3.5) / 0.1)
    snf_steps = round((snf - 8.5) / 0.1)

    rate = BASE_RATE
    rate += Decimal(fat_steps) * FAT_STEP_VALUE
    rate += Decimal(snf_steps) * SNF_STEP_VALUE
    return rate.quantize(Decimal("0.01"))

def payout(litres, fat, snf):
    """Calculate payment using exact decimal money."""
    rate = rate_per_litre(fat, snf)
    amount = Decimal(str(litres)) * rate
    return amount.quantize(Decimal("0.01"))

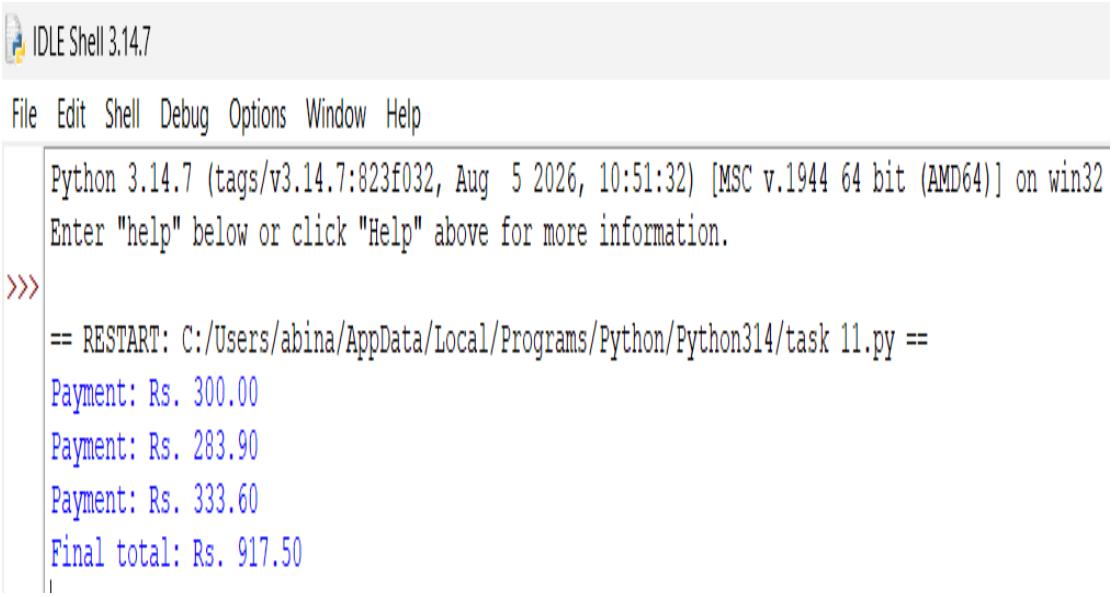
entries = [
    (10, 3.5, 8.5),
    (8.5, 4.2, 8.7),
    (12, 3.1, 8.3)
]

total = Decimal("0.00")

for litres, fat, snf in entries:
    amount = payout(litres, fat, snf)
    total += amount
    print("Payment: Rs.", amount)

print("Final total: Rs.", total)
```

Output:



Task 12 — Seasonal Rates and Bonus Calculation

Aim: To calculate milk payments using seasonal rate cards, two-decimal readings and a ten-day supply bonus.

Algorithm:

1. Start.
2. Define the standard fat and SNF values.
3. Define the regular and seasonal rate cards.
4. Read or retrieve the ten-day milk entries.
5. Select the required season.
6. Validate the milk quantity, fat and SNF values.
7. Calculate the rate for each entry using Decimal.
8. Calculate the individual payments and base total.
9. Calculate the total milk quantity supplied.
10. STOP

Input:

task 12.py - C:/Users/abina/AppData/Local/Programs/Python/Python314/task 12.py (3.14.7)

File Edit Format Run Options Window Help

```
from decimal import Decimal
BASE_RATE = Decimal("30.00")
FAT_STEP_VALUE = Decimal("0.40")
SNF_STEP_VALUE = Decimal("0.30")
SEASONAL_BASE_RATE = Decimal("32.00")
SEASONAL_FAT_STEP_VALUE = Decimal("0.45")

BONUS_LIMIT = Decimal("500")
BONUS_PER_LITRE = Decimal("0.50")


def rate_per_litre(fat, snf, month):
    """Calculate milk rate based on the month."""
    if month in ["April", "May", "June"]:
        base = SEASONAL_BASE_RATE
        fat_value = SEASONAL_FAT_STEP_VALUE
    else:
        base = BASE_RATE
        fat_value = FAT_STEP_VALUE

    fat_steps = round((fat - 3.5) / 0.1)
    snf_steps = round((snf - 8.5) / 0.1)

    rate = base
    rate += Decimal(fat_steps) * fat_value
    rate += Decimal(snf_steps) * SNF_STEP_VALUE

    return rate.quantize(Decimal("0.01"))


def payout(litres, fat, snf, month):
    """Calculate payment for one milk entry."""
    rate = rate_per_litre(fat, snf, month)
    amount = Decimal(str(litres)) * rate
    return amount.quantize(Decimal("0.01"))


def ten_day_total(entries, month):
    """Calculate total payment and eligible volume bonus."""
    total_litres = sum(
        (Decimal(str(entry[0])) for entry in entries),
        Decimal("0")
    )

    total_payment = Decimal("0.00")


    for litres, fat, snf in entries:
        total_payment += payout(litres, fat, snf, month)

    if total_litres > BONUS_LIMIT:
        bonus = total_litres * BONUS_PER_LITRE
    else:
        bonus = Decimal("0.00")

    bonus = bonus.quantize(Decimal("0.01"))
    total_payment += bonus

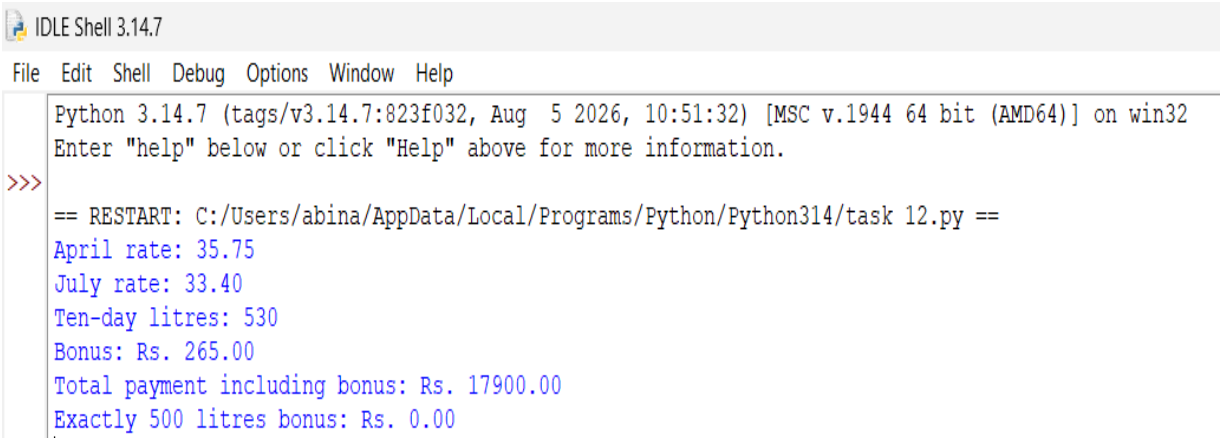
    return total_litres, bonus, total_payment
print("April rate:", rate_per_litre(4.2, 8.7, "April"))
print("July rate:", rate_per_litre(4.2, 8.7, "July"))
entries = [
    (200, 3.5, 8.5),
    (180, 4.2, 8.7),
    (150, 3.5, 8.5)
]

litres, bonus, total = ten_day_total(entries, "April")

print("Ten-day litres:", litres)
print("Bonus: Rs.", bonus)
print("Total payment including bonus: Rs.", total)
entries_500 = [(500, 3.5, 8.5)]
litres, bonus, total = ten_day_total(entries_500, "April")

print("Exactly 500 litres bonus: Rs.", bonus)
```

Output:


IDLE Shell 3.14.7
File Edit Shell Debug Options Window Help
Python 3.14.7 (tags/v3.14.7:823f032, Aug 5 2026, 10:51:32) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
== RESTART: C:/Users/abina/AppData/Local/Programs/Python/Python314/task 12.py ==
April rate: 35.75
July rate: 33.40
Ten-day litres: 530
Bonus: Rs. 265.00
Total payment including bonus: Rs. 17900.00
Exactly 500 litres bonus: Rs. 0.00